

Data Wrangling 2: Cleaning, Merging & GroupBy



Suman

24 Feb, 2026 At 8:00 PM

Notes

Module-1

Recommended

Resource



Associated Lectures (1)



Description



Discussions

Session 4.2: Lecture Notes

PPT File: [Click Here](#)

Colab File: [Click Here](#)

Data Wrangling 2: Cleaning, Merging & GroupBy

Program: AIM101 -- Artificial Intelligence & Machine Learning Foundation **Institution:** Vishlesan i-Hub IIT Patna x Masai School **Session Type:** Instructor (Practical, Hands-on) **Prerequisites:** Session 4.1 (Pandas DataFrames: Creation, Indexing & Selection)

Mental Map

```
Session 4.2: Data Wrangling – Cleaning, Merging & GroupBy
|
+-- A. HANDLING MISSING VALUES
|   +-- What is NaN? (np.nan, None -> NaN)
|   +-- Detecting: .isna(), .notna(), .isna().sum()
|   +-- .info() Non-Null Count as first check
|   +-- Dropping: .dropna(subset=, thresh=)
|   +-- Filling: .fillna(value), .fillna(mean/median), .ffill()
|   +-- Decision framework: when to drop vs when to fill
|
+-- B. HANDLING DUPLICATES
|   +-- Why duplicates are dangerous (silent inflation)
|   +-- Detecting: .duplicated(), .duplicated(subset=, keep=)
|   +-- Removing: .drop_duplicates(subset=, keep=)
|   +-- The subset parameter: unique identifier columns
|
+-- C. DATA TYPE CONVERSION
|   +-- The hidden type problem (object dtype for numbers/dates)
|   +-- .astype() – simple, direct casting (clean data only)
```

```

|   +-- pd.to_numeric(errors='coerce') - safe numeric conversion
|   +-- pd.to_datetime(format='mixed') - safe date conversion
|   +-- errors parameter: 'raise' vs 'coerce' vs 'ignore'
|
+-- D. STRING OPERATIONS (.str accessor)
|   +-- .str.strip() - remove whitespace
|   +-- .str.lower(), .str.upper(), .str.title() - case standardization
|   +-- .str.replace(), .str.contains() - search and replace
|   +-- .str.split(), .str.len() - structure and validation
|   +-- 3-step pipeline: strip -> case -> specific fixes
|
+-- E. MESSY DATA PATTERNS
|   +-- Tidy data: 3 rules (Wickham, 2014)
|   +-- Wide format vs Long format
|   +-- pd.melt() - wide to long (unpivot)
|   +-- .pivot() - long to wide
|   +-- Pattern recognition (awareness level)
|
+-- F. COMBINING DATAFRAMES: CONCAT
|   +-- pd.concat() - "dumb gluing" (no key matching)
|   +-- Vertical stacking (axis=0): more rows, same columns
|   +-- Horizontal stacking (axis=1): same rows, more columns
|   +-- ignore_index=True - prevent duplicate indices
|   +-- join='inner' vs join='outer' for mismatched columns
|
+-- G. COMBINING DATAFRAMES: MERGE/JOIN
|   +-- pd.merge() - VLOOKUP on steroids
|   +-- on=, left_on=, right_on= for key columns
|   +-- INNER join: only matching rows (default)
|   +-- LEFT join: all left rows + matching right (most common)
|   +-- RIGHT join: all right rows + matching left
|   +-- OUTER join: everything from both sides
|   +-- indicator=True for debugging merges
|   +-- Many-to-many danger: Cartesian product / row explosion
|
+-- H. GROUPBY & AGGREGATION
|   +-- Split-Apply-Combine pattern
|   +-- .groupby('col')['val'].sum/mean/count()
|   +-- .agg() - multiple aggregations, named aggregations
|   +-- .transform() - broadcast group values back to rows
|   +-- .pivot_table() - cross-tabulation summaries
|   +-- .reset_index() - always call after groupby

```

The Big Picture

The Data Wrangling Pipeline

+-----+		+-----+		+-----+		+-----+
DETECT	-->	CLEAN	-->	COMBINE	-->	ANALYZE
.info()		.fillna()		pd.concat()		.groupby()
.isna().sum()		.dropna()		pd.merge()		.agg()
.duplicated()		.drop_duplicates		(4 join types)		.transform()
.dtypes		pd.to_numeric()				.pivot_table

.describe()	pd.to_datetime()		
	.str.strip()		
	.str.lower()		
+-----+	+-----+	+-----+	+-----+

Session 4.1 vs Session 4.2

Session 4.1	Session 4.2
Create DataFrames	Clean messy DataFrames
Load CSV files	Fix data types, strings, missing values
Select, filter, sort	Combine multiple DataFrames
Work with clean data	Work with real-world messy data
Single DataFrame operations	Multi-DataFrame pipelines
Data Consumer	Data Wrangler

Key Insight: Data scientists spend 60-80% of their time on data wrangling. Everything in this session is what fills that 80%.

TOPIC A: Handling Missing Values

A.1 What Is NaN?

- **NaN** = Not a Number -- Pandas' universal marker for "this value is missing"
- NaN comes from NumPy: `np.nan`
- NaN is NOT zero. NaN is NOT an empty string. NaN is NOT False.
- NaN is the **absence** of a value

The strangest property:

```
np.nan == np.nan    # Returns False!
```

NaN in calculations:

- `5 + NaN = NaN` (any math with NaN produces NaN -- the "poison" property)
- Pandas `.mean()`, `.sum()` **ignore NaN by default** (unlike raw NumPy)

Where NaN appears:

- Loading a CSV with empty cells
- Merging DataFrames with mismatched keys

- Creating DataFrames from dictionaries with unequal-length lists
- `None` in a numeric column auto-converts to NaN

FORM ANALOGY:

```

+-----+
| Patient Intake Form      |
+-----+
| Name:    Rajesh Kumar    | <- filled
| Age:     45              | <- filled
| Blood:   A+              | <- filled
| Allergy: _____     | <- BLANK = NaN
| Weight:  _____     | <- BLANK = NaN
+-----+
Blank != "None"    Blank != "0"    Blank = "Not recorded"

```

A.2 Detecting Missing Values

Step 1: Bird's-eye view with `.info()`

```

df.info()
# Look at the "Non-Null Count" column
# If a column has 10 entries but only 8 non-null -> 2 missing values

```

Step 2: Boolean mask with `.isna()`

```

df['age'].isna()      # Series of True/False for one column
df.isna()             # DataFrame of True/False for all columns

```

Step 3: Count missing values

```

# The Data Quality Snapshot -- memorize this pattern
df.isna().sum()          # Count of missing per column
df.isna().sum().sum()    # Total missing in entire DataFrame
df.isna().mean() * 100   # Percentage missing per column

```

Inverse: `.notna()` returns True where values ARE present

```

df[df['age'].notna()]    # Only rows where age exists

```

A.3 Dropping Missing Values -- `.dropna()`

Syntax	Behavior
<code>df.dropna()</code>	Drop any row with ANY NaN (aggressive!)
<code>df.dropna(subset=['age'])</code>	Drop rows only where 'age' is NaN (surgical)

Syntax	Behavior
<code>df.dropna(thresh=5)</code>	Keep rows with at least 5 non-null values
<code>df.dropna(axis=1)</code>	Drop COLUMNS with any NaN (rarely used)

Key behavior: `.dropna()` returns a NEW DataFrame -- the original is unchanged.

Pitfall: Blind `df.dropna()` can lose half your data. Always check the shape before and after.

A.4 Filling Missing Values -- `.fillna()`

Strategy	Code	When to Use
Fill with constant	<code>df['name'].fillna('Unknown')</code>	Placeholder for text columns
Fill with mean	<code>df['age'].fillna(df['age'].mean())</code>	Numerical data, symmetric distribution
Fill with median	<code>df['age'].fillna(df['age'].median())</code>	Numerical data, robust to outliers
Forward fill	<code>df['temp'].ffill()</code>	Time series (use previous value)
Backward fill	<code>df['temp'].bfill()</code>	Time series (use next value)

A.5 Decision Framework: Drop vs Fill

```

IS THE COLUMN CRITICAL?
+-- YES (e.g., patient blood type)
|   +-- Can you look it up from another source?
|       +-- YES -> Look it up and fill
|       +-- NO  -> Flag the row, do NOT guess
|
+-- NO (e.g., middle name)
    +-- What % is missing?
        +-- < 5% -> DROP rows with .dropna(subset=[...])
        +-- 5-30% -> FILL with mean/median/mode
        +-- > 50% -> DROP the entire COLUMN

```

There is NO universal correct answer. The right strategy depends on the domain, the column's importance, and the percentage missing.

TOPIC B: Handling Duplicates

B.1 Why Duplicates Are Dangerous

Duplicates do not cause errors -- your code runs perfectly, your numbers look reasonable. But they are **silently wrong** because some records are counted more than once.

Real-world consequences:

- **Finance:** Bank processes the same transaction twice -- customer charged double
- **E-commerce:** Duplicate order ships twice -- company absorbs return costs
- **Healthcare:** Duplicate claims flagged as potential fraud
- **Analytics:** Survey response counted twice -- statistics inflated

B.2 Detecting Duplicates -- `.duplicated()`

Syntax	What It Marks as True
<code>df.duplicated()</code>	Second (and subsequent) occurrences
<code>df.duplicated(keep='last')</code>	All except the LAST occurrence
<code>df.duplicated(keep=False)</code>	ALL duplicates (including first occurrence)
<code>df.duplicated(subset=['patient_id'])</code>	Based on specific column(s) only

```
# Count duplicates
df.duplicated().sum()

# Show ALL rows involved in duplication (for investigation)
dup_mask = df.duplicated(subset=['patient_id'], keep=False)
df[dup_mask]
```

Key insight: Without `subset`, two rows must be identical in EVERY column. With `subset=['patient_id']`, only the specified column(s) must match.

B.3 Removing Duplicates -- `.drop_duplicates()`

```
# Keep first occurrence (default)
df.drop_duplicates(subset=['patient_id'], keep='first')

# Keep last occurrence
df.drop_duplicates(subset=['patient_id'], keep='last')

# Remove ALL copies (keep none)
df.drop_duplicates(subset=['patient_id'], keep=False)
```

The `keep` decision depends on domain context:

```
keep='first' -> Trust the earliest/original entry
keep='last'  -> Trust the most recent/updated entry
keep=False   -> Remove ALL duplicates, investigate manually
```

Key behavior: `.drop_duplicates()` returns a NEW DataFrame -- the original is unchanged.

Best practice: Always LOOK at duplicates before blindly removing them. Sometimes "duplicates" have meaningful differences (e.g., an updated email, a corrected address).

TOPIC C: Data Type Conversion

C.1 The Hidden Type Problem

- `bill_amount` LOOKS numeric ('15000', '22500.50') but is stored as **strings** (dtype = `object`)
- `admission_date` LOOKS like dates but Pandas sees them as **plain text**
- You CANNOT do math on strings: `'15000' + '22500'` gives `'1500022500'` (concatenation!), not `37500`
- You CANNOT sort date strings correctly: `'15-01-2024'` sorts BEFORE `'2024-01-15'` alphabetically

One bad apple spoils the bunch: If even ONE value in a column is non-numeric (like 'N/A'), Pandas stores the ENTIRE column as strings.

C.2 Three Conversion Tools

Tool	Purpose	Best For	Key Feature
<code>.astype()</code>	Simple, direct type casting	Clean data with no errors	Fast but crashes on bad data
<code>pd.to_numeric()</code>	String to Number conversion	Columns with messy entries ('N/A', '-')	<code>errors</code> parameter handles bad values
<code>pd.to_datetime()</code>	String to DateTime conversion	Date columns in various formats	<code>format='mixed'</code> handles inconsistent formats

C.3 The `errors` Parameter

<code>errors=</code>	Behavior	When to Use
<code>'raise'</code>	Crashes with ValueError (default)	When you WANT to know about bad data
<code>'coerce'</code>	Converts failures to NaN	Production standard -- convert what you can
<code>'ignore'</code>	Leaves column unchanged	Rarely useful -- avoid in practice

C.4 Conversion Patterns

Numeric conversion (safe):

```
df['bill_amount'] = pd.to_numeric(df['bill_amount'], errors='coerce')
# 'N/A' becomes NaN; '15000' becomes 15000.0
# Check: df['bill_amount'].isna().sum() -> count of failures
```

Date conversion (handles mixed formats):

```
df['date'] = pd.to_datetime(df['date'], format='mixed', dayfirst=True)
# Handles both '2024-01-15' and '15-01-2024' in the same column
```

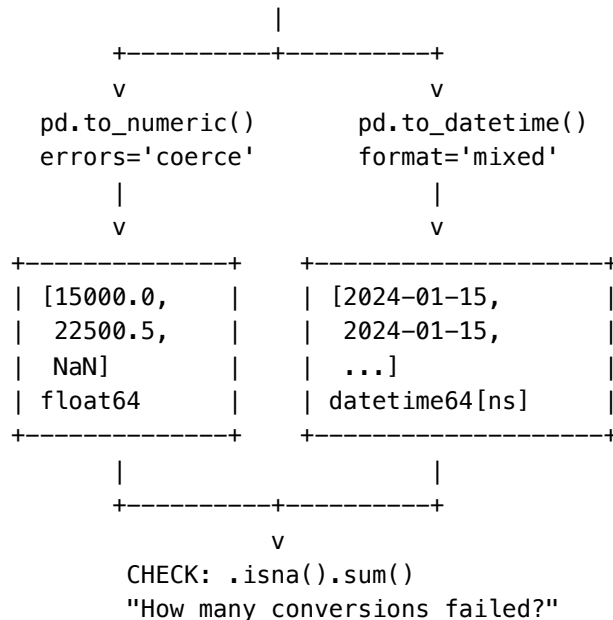
Simple casting (clean data only):

```
df['age'] = df['age'].astype(int)      # Only if NO NaN values
df['price'] = df['price'].astype(float) # Only if ALL values are numeric
```

C.5 Type Conversion Pipeline

RAW DATA (all strings)

```
+-----+
| bill_amount: ['15000', '22500.50', 'N/A'] | <- object dtype
| admission_date: ['2024-01-15', '15-01-2024'] | <- object dtype
+-----+
```



TOPIC D: String Operations (.str Accessor)

D.1 The Text Mess

If you group by department, Pandas sees 'Cardiology' and 'cardiology' as TWO different departments. Your patient counts, revenue sums, and averages will all be WRONG.

patients['name']:		patients['department']:	
'Rajesh Kumar'	<- Title case	'Cardiology'	<- Capital C
' Priya Sharma '	<- SPACES!	'cardiology'	<- lowercase c
'ANITA DESAI'	<- ALL CAPS	'Orthopedics'	<- Capital O
'vikram patel'	<- all lower	'orthopedics'	<- lowercase o
None	<- Missing!		

D.2 The .str Accessor -- String Methods for Entire Columns

Method	What It Does	Example
<code>.str.lower()</code>	All lowercase	'ANITA DESAI' -> 'anita desai'
<code>.str.upper()</code>	All uppercase	'vikram patel' -> 'VIKRAM PATEL'
<code>.str.title()</code>	Title Case	'vikram patel' -> 'Vikram Patel'
<code>.str.strip()</code>	Remove leading/trailing whitespace	' Priya Sharma ' -> 'Priya Sharma'
<code>.str.replace(old, new)</code>	Replace substrings	Replace targeted text in every row
<code>.str.contains(pattern)</code>	Boolean: does it contain?	Returns True/False per row
<code>.str.split(sep)</code>	Split into parts	'Rajesh Kumar' -> ['Rajesh', 'Kumar']
<code>.str.len()</code>	Length of each string	'Rajesh Kumar' -> 12

Key insight: `.str` methods automatically **skip NaN values** -- no crash on missing data.

D.3 The 3-Step Cleaning Pipeline

STEP 1: Strip whitespace	' Priya Sharma '	->	'Priya Sharma'
STEP 2: Standardize case	'ANITA DESAI'	->	'Anita Desai'
	'vikram patel'	->	'Vikram Patel'
STEP 3: Fix specific values	(optional -- for known typos or aliases)		

One-liner chaining:

```
# For name columns: strip + title case
df['name'] = df['name'].str.strip().str.title()

# For categorical columns: normalize to lowercase
df['department'] = df['department'].str.lower()
```

Why strip FIRST: ' Priya ' and 'Priya' remain different after case standardization if you do not strip first.

D.4 Filtering with .str.contains()

```
# Case-insensitive substring search
cardio_mask = df['department'].str.contains('cardio', case=False)
df[cardio_mask]

# Alternative: lowercase first, then contains
df[df['department'].str.lower().str.contains('cardio')]
```

TOPIC E: Messy Data Patterns

E.1 Tidy Data Principles (Wickham, 2014)

Three rules that define "tidy data":

Each variable forms a column -- one piece of information per column

Each observation forms a row -- one entity/event per row

Each type of observational unit forms a table -- customers in one table, orders in another

Tidy data is the input format that Pandas expects. If your data is messy, your FIRST job is to make it tidy.

E.2 Common Messy Pattern: Wide vs Long

WIDE FORMAT (messy):

Student	Math	Science	English
Priya	85	92	78
Rahul	76	88	91

Column headers "Math", "Science", "English" are VALUES of a variable "Subject"

LONG/TIDY FORMAT (clean):

Student	Subject	Score
Priya	Math	85
Priya	Science	92
Priya	English	78
Rahul	Math	76
Rahul	Science	88
Rahul	English	91

Student	Subject	Score
Priya	Math	85
Priya	Science	92
Priya	English	78
Rahul	Math	76
Rahul	Science	88
Rahul	English	91

E.3 Reshaping Tools

Wide to Long: `pd.melt()`

```
scores_long = pd.melt(
    scores_wide,
    id_vars=['student'],      # Columns to KEEP as-is
    var_name='subject',      # Name for new "variable" column
    value_name='score'       # Name for new "value" column
)
```

Long to Wide: `.pivot()`

```
scores_wide = scores_long.pivot(
    index='student',          # Row labels
    columns='subject',        # Column headers
    values='score'            # Cell values
)
```

Key takeaway: The number of data points is the SAME -- only the SHAPE changes. Recognize the pattern so you know WHAT tool to reach for.

E.4 Tidy Data Checklist

ASK YOURSELF:

```
+-----+
| Is each column ONE variable? |
|   -> If column headers are values -> pd.melt() |
| | | | |
| Is each row ONE observation? |
|   -> If multiple observations per row -> pd.melt() |
| | | | |
| Is each table ONE type of thing? |
|   -> If mixed (orders + customers) -> split tables |
+-----+
```

TOPIC F: Combining DataFrames -- Concat

F.1 What Is Concat?

`pd.concat()` **stacks** DataFrames together -- "dumb gluing." It does NOT match on any key column.

VERTICAL STACK (axis=0):

```
+-----+
| January |
| (5 rows) |
+-----+
| February |
| (5 rows) |
+-----+
10 rows x same columns
```

HORIZONTAL STACK (axis=1):

```
+-----+-----+
| Orders | Revenue |
| (5 rows) | (5 rows) |
+-----+-----+
5 rows x more columns
```

F.2 pd.concat() Syntax & Parameters

```
# Vertical stacking (default, axis=0)
combined = pd.concat([df1, df2], ignore_index=True)

# Horizontal stacking (axis=1)
combined = pd.concat([df1, df2], axis=1)
```

Parameter	Default	What It Does
axis	0	0 = stack vertically (more rows), 1 = horizontally (more columns)
ignore_index	False	If True, resets index to 0, 1, 2, ...
join	'outer'	'outer' keeps all columns; 'inner' keeps only shared columns

F.3 The #1 Gotcha: Duplicate Indices

```
WITHOUT ignore_index:      WITH ignore_index=True:
  name  sales              name  sales
0 Alice   100              0 Alice   100
1 Bob    200              1 Bob    200
0 Charlie 150  <- DUPLICATE 2 Charlie 150  <- CLEAN
1 Diana   180  INDEX!      3 Diana   180
```

Rule: Always use `ignore_index=True` when stacking rows.

F.4 Mismatched Columns

```
# Outer join (default) -- keeps ALL columns, fills NaN where missing
pd.concat([df_a, df_b], ignore_index=True)
```

```
# Inner join -- keeps ONLY shared columns
pd.concat([df_a, df_b], ignore_index=True, join='inner')
```

TOPIC G: Combining DataFrames -- Merge/Join

G.1 The Lookup Problem

Your `orders` table has `customer_id` but not the customer's name or city. That information lives in a SEPARATE `customers` table. You need to reach into the other table and PULL that information in.

orders:			customers:		
order_id	customer_id		customer_id	name	city
ORD001	C001	----->	C001	Priya Sharma	Mumbai
ORD002	C002	----->	C002	Rahul Verma	Delhi
ORD003	C003	----->	C003	Anita Desai	Bangalore

Result: Each order now has customer name and city attached!

G.2 pd.merge() Syntax

```
# Basic merge on shared column name
pd.merge(orders, customers, on='customer_id')

# When column names differ between tables
pd.merge(orders, customers, left_on='cust_id', right_on='customer_id')

# With indicator for debugging
pd.merge(orders, customers, on='customer_id', how='outer', indicator=True)
```

The `indicator=True` parameter adds a `_merge` column:

- `both` -- matched in BOTH tables
- `left_only` -- only in left table
- `right_only` -- only in right table

G.3 The Four Join Types

```
INNER JOIN (how='inner') -- DEFAULT
Only rows with matching keys in BOTH tables survive.
"Only the overlap"
```

LEFT JOIN (how='left')

ALL rows from left table + matching from right. Non-matches get NaN.

"Everything from left, look up from right"

RIGHT JOIN (how='right')

ALL rows from right table + matching from left. Non-matches get NaN.

"Everything from right, look up from left"

OUTER JOIN (how='outer')

ALL rows from BOTH tables. Non-matches filled with NaN.

"Everything from everywhere"

Visual:

1. INNER JOIN

+---+ +---+

| L |X| R |

| |X| |

+---+ +---+

Only overlap

2. LEFT JOIN

+---+ +---+

|LLL|X| R |

|LLL|X| |

+---+ +---+

All left +
matching right

3. RIGHT JOIN

+---+ +---+

| L |X|RRR|

| |X|RRR|

+---+ +---+

All right +
matching left

4. OUTER JOIN

+---+ +---+

|LLL|X|RRR|

|LLL|X|RRR|

+---+ +---+

Everything

G.4 Which Join to Use?

Join Type	When to Use	Example
Inner	Only want rows that match in both tables	"Show orders with known customers only"
Left	Keep ALL rows from primary table (most common)	"All orders, enriched with customer info"
Right	Keep ALL rows from lookup table	"All customers, even if no orders"
Outer	Find what is MISSING (diagnostic)	"What orders lack customers? What customers lack orders?"

Left join is by far the most commonly used in practice.

G.5 The Many-to-Many Danger

If the key column is NOT unique in BOTH tables, you get a **Cartesian product** (row explosion):

LEFT (3 rows) x RIGHT (3 rows) = RESULT (9 rows!)

Prevention rules:

```
# ALWAYS check uniqueness before merging
df['key'].is_unique

# ALWAYS compare shape before and after
print(f"Left: {len(left)}, Right: {len(right)}, Result: {len(merged)}")
```

TOPIC H: GroupBy & Aggregation

H.1 The Split-Apply-Combine Pattern

The question: "What are the total sales by city?" "Average order value by segment?"

All follow the same pattern:

Split the data into groups (by city, by segment, by category)

Apply a function to each group (sum, mean, count)

Combine the results into a summary table

ORIGINAL DATA:	SPLIT -> APPLY -> COMBINE
<pre>+-----+-----+ City Sales +-----+-----+ Mumbai 100 Delhi 80 Mumbai 200 Delhi 120 Mumbai 150 +-----+-----+ 5 rows</pre>	<pre>Mumbai: [100, 200, 150] -> sum = 450 Delhi: [80, 120] -> sum = 200 RESULT: +-----+-----+ City Sales +-----+-----+ Mumbai 450 Delhi 200 +-----+-----+ 2 rows (one per group)</pre>

H.2 .groupby() Basics

```
# Single column groupby
df.groupby('city')['sales'].sum()

# Multi-column groupby
df.groupby(['city', 'segment'])['sales'].sum()

# ALWAYS call .reset_index() for a clean DataFrame
df.groupby('city')['sales'].sum().reset_index()
```

Common aggregation functions:

Function	What It Does	Example Use Case
<code>.sum()</code>	Total	Total sales per city
<code>.mean()</code>	Average	Average order value per segment
<code>.count()</code>	Count of non-null values	Number of orders per customer
<code>.min()</code>	Smallest value	Cheapest order per category
<code>.max()</code>	Largest value	Most expensive order per category
<code>.median()</code>	Middle value	Median salary per department
<code>.nunique()</code>	Unique value count	Unique products per category

H.3 .agg() -- Multiple Aggregations

```
# Multiple aggregations on one column
df.groupby('city')['sales'].agg(['sum', 'mean', 'count'])

# Different aggregations on different columns (dictionary syntax)
df.groupby('city').agg({
    'sales': 'sum',
    'quantity': 'mean',
    'order_id': 'count'
})

# Named aggregations (recommended modern syntax)
df.groupby('city').agg(
    total_sales=('sales', 'sum'),
    avg_quantity=('quantity', 'mean'),
    order_count=('order_id', 'count')
).reset_index()
```

Named aggregations produce clean, descriptive column names directly. Use this syntax for new code.

H.4 .transform() vs .agg()

	<code>.agg()</code>	<code>.transform()</code>
Output rows	One row per group (REDUCES)	Same rows as input (BROADCASTS)
Use for	Summary tables, reports	Adding group context to individual rows
Analogy	Writing section average on the BOARD	Writing section average on each STUDENT'S paper

```
# .agg() -- REDUCES: 20 rows -> N groups
sales_by_city = df.groupby('city')['sales'].agg('sum').reset_index()
```

```
# .transform() -- BROADCASTS: 20 rows -> 20 rows (same size)
df['city_total'] = df.groupby('city')['sales'].transform('sum')
df['pct_of_city'] = (df['sales'] / df['city_total'] * 100).round(1)
```

H.5 .pivot_table() -- Cross-Tabulation Summaries

```
pd.pivot_table(
    df,
    values='sales',           # What to aggregate
    index='city',             # Row labels
    columns='category',       # Column headers
    aggfunc='sum',            # How to aggregate
    fill_value=0              # Replace NaN with 0
)
```

With totals:

```
pd.pivot_table(
    df,
    values='sales',
    index='city',
    columns='category',
    aggfunc='sum',
    fill_value=0,
    margins=True,             # Add row and column totals
    margins_name='TOTAL'
)
```

When to use pivot_table vs groupby:

- **groupby:** Flat summary (city -> total_sales)
- **pivot_table:** Cross-tabulation (city rows x category columns -> sales values)

groupby output:

city	sales
Mumbai	15000
Delhi	12000
Bangalore	8000

Flat -- one column

pivot_table output:

city	Books	Clothing	Electr.
Mumbai	599	499	4398
Delhi	599	2499	3499
Bangalore	0	0	1299

Cross-tab -- categories as columns

KEY FRAMEWORKS

Drop vs Fill Decision Framework

Scenario	Recommended Action
Column is critical (blood type) + cannot look up	Flag the row -- do NOT guess
Column is critical + can look up from another source	Fill from the source
Column is non-critical + <5% missing	Drop rows with <code>.dropna(subset=[...])</code>
Column is non-critical + 5-30% missing	Fill with mean/median/mode
Column has >50% missing	Drop the entire column

Which Join Type to Use

Situation	Join Type	Why
Only want matched records	<code>inner</code>	Strict -- no NaN in result
Enrich primary table with lookups	<code>left</code>	Most common -- keep all primary rows
Find unmatched records (debug)	<code>outer + indicator=True</code>	See what is missing on each side
Reverse perspective of left join	<code>right</code>	Swap arguments and use <code>left</code> instead

concat vs merge

	<code>pd.concat()</code>	<code>pd.merge()</code>
Logic	Dumb gluing -- stacks DataFrames	Smart lookup -- matches on key columns
Use when	Same structure, different data batches	Different tables with a shared key
Analogy	Stacking cafeteria trays	VLOOKUP in Excel
Key matching	None -- just stacks	Yes -- uses <code>on=</code> parameter

.agg() vs .transform() Quick Decision

Question	Use
"What is the total sales per city?" (summary)	<code>.agg()</code>
"What % of city sales is each order?" (context)	<code>.transform()</code>

Question	Use
Need fewer rows (one per group)	<code>.agg()</code>
Need same rows with group info added	<code>.transform()</code>

COMMON ERRORS QUICK REFERENCE

#	Error	Cause	Fix
1	<code>ValueError: Unable to parse string "N/A"</code>	<code>pd.to_numeric()</code> with default <code>errors='raise'</code>	Use <code>errors='coerce'</code>
2	String concatenation instead of addition	Column stored as <code>object</code> (string), not numeric	Convert with <code>pd.to_numeric(errors='coerce')</code> first
3	<code>'Cardiology'</code> and <code>'cardiology'</code> counted separately	Inconsistent casing in text column	<code>df['col'] = df['col'].str.lower()</code>
4	<code>.duplicated()</code> returns 0 but duplicates exist	Whitespace difference hides duplicates	Use <code>subset=['id_column']</code> instead of checking all columns
5	Duplicate index values after <code>pd.concat()</code>	Forgot <code>ignore_index=True</code>	<code>pd.concat([df1, df2], ignore_index=True)</code>
6	Row explosion after <code>pd.merge()</code>	Many-to-many merge (non-unique key in both tables)	Check <code>df['key'].is_unique</code> before merging
7	GroupBy result has grouped column as index	Did not call <code>.reset_index()</code>	Chain <code>.reset_index()</code> after <code>.agg()</code>
8	<code>NaN</code> forced integer column to float64	Pandas upcasts int to float when <code>NaN</code> is present	Expected behavior; use <code>Int64</code> nullable type if needed
9	<code>.fillna()</code> or <code>.dropna()</code> did not change the DataFrame	These methods return NEW DataFrames	Assign: <code>df = df.dropna()</code> or <code>df['col'] = df['col'].fillna(0)</code>
10	<code>pd.to_datetime()</code> parsing wrong dates	Ambiguous dates (01-02-2024: Jan 2 or Feb 1?)	Use <code>dayfirst=True</code> for DD-MM-YYYY format

#	Error	Cause	Fix
11	<code>.str</code> method crashed with <code>AttributeError</code>	Column contains non-string types	Ensure column is <code>object</code> dtype; <code>.str</code> only works on strings
12	<code>merge()</code> produced unexpected NaN columns	Column name exists in both DataFrames	Use <code>suffixes=('_left', '_right')</code> parameter
13	<code>pivot_table()</code> shows NaN for missing combinations	No data for that row-column combination	Use <code>fill_value=0</code> parameter

KEY TAKEAWAYS

MISSING DATA DETECTION: Always start with `.info()` and `.isna().sum()` to understand what is missing and where. The percentage of missing values per column determines your handling strategy.

MISSING DATA HANDLING: Use `.fillna()` for imputation (mean, median, or domain-appropriate constant) and `.dropna()` for removal. Choose based on context -- a missing blood type cannot be "averaged," but a missing weight can.

DUPLICATE REMOVAL: Use `.duplicated(subset=['id_column'])` to detect and `.drop_duplicates()` to remove. Always investigate duplicates before blindly removing them -- whitespace and casing differences can hide obvious duplicates.

DATA TYPE CONVERSION: Use `pd.to_numeric(errors='coerce')` for safe numeric conversion and `pd.to_datetime(format='mixed')` for dates. The `errors='coerce'` pattern converts what it can and turns failures into NaN for later review.

STRING CLEANING: The `.str` accessor applies string methods to entire columns at once. The 3-step pipeline (strip whitespace, standardize case, fix specifics) handles 90% of text data quality issues. Inconsistent casing is an invisible data quality killer.

CONCATENATION: `pd.concat()` stacks DataFrames vertically (more rows) or horizontally (more columns). Always use `ignore_index=True` to avoid duplicate index values.

MERGING / JOINS: `pd.merge()` combines DataFrames on shared key columns using four join types: inner (intersection), left (all primary + matched lookup), right (all lookup + matched primary), and outer (everything). Left join is the most common in practice.

GROUPBY + AGG: The split-apply-combine pattern (`.groupby().agg().reset_index()`) is the most powerful single operation in Pandas for summarization. Named aggregations produce the cleanest output.

PIVOT TABLES & TRANSFORM: `.pivot_table()` creates cross-tabulation summaries (rows x columns). `.transform()` broadcasts group-level values back to individual rows, enabling relative analysis like "what percentage of city sales does this order represent?"

NEXT SESSION PREVIEW

Session 4.3: Data Quality -- Principles & Industry Horror Stories

Topic	What You Will Learn
Data Quality Dimensions	Accuracy, Completeness, Consistency, Timeliness
Garbage In, Garbage Out	Even the best ML model fails on bad data
Industry Horror Stories	IBM Watson Oncology, JPMorgan London Whale, UK Post Office Horizon
Data Governance Basics	Documentation, validation, regulatory awareness

Why it matters:

- This session you learned HOW to clean data
- Next session you learn WHY data quality is the difference between a working model and a billion-dollar disaster

Key cases:

- **IBM Watson Oncology:** AI gave unsafe cancer treatment recommendations because training data was based on hypothetical cases, not real patients
- **JPMorgan London Whale:** \$6 BILLION loss partly caused by a copy-paste error in a spreadsheet risk model
- **UK Post Office Horizon:** Faulty software data sent 700+ innocent sub-postmasters to prison over 20 years

Note: Session 4.3 is a Professor session -- NO CODE. Pure theory, case studies, and frameworks.

Footer: Vishlesan i-Hub IIT Patna x Masai School